МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторные работы
по курсу «Информационный поиск»

Выполнил: Калюжный Михаил Сергеевич
Группа: М8О-408Б-22
Преподаватель: Кухтичев Антон Алексеевич

Москва, 2026

Содержание

1 Лабораторная работа №1. Добыча корпуса документов **2**
1.1 Задание . . . 2
1.2 Метод решения . . . 2
1.3 Результаты . . . 2
1.4 Выводы . . . 2

2 Лабораторная работа №2. Поисковый робот **3**
2.1 Задание . . . 3
2.2 Метод решения . . . 3
2.3 Результаты . . . 4
2.4 Выводы . . . 4

3 Лабораторная работа №3. Токенизация **5**
3.1 Задание . . . 5
3.2 Метод решения . . . 5
3.3 Результаты . . . 5
3.4 Выводы . . . 5

4 Лабораторная работа №4. Закон Ципфа **6**
4.1 Задание . . . 6
4.2 Метод решения . . . 6
4.3 Результаты . . . 6
4.4 Выводы . . . 7

5 Лабораторная работа №5. Стемминг **8**
5.1 Задание . . . 8
5.2 Метод решения . . . 8
5.3 Результаты . . . 8
5.4 Выводы . . . 8

6 Лабораторная работа №6. Булев индекс **9**
6.1 Задание . . . 9
6.2 Метод решения . . . 9
6.3 Результаты . . . 9
6.4 Выводы . . . 9

7 Лабораторная работа №7. Булев поиск **10**
7.1 Задание . . . 10
7.2 Метод решения . . . 10
7.2.1 Парсер запросов . . . 10
7.2.2 Выполнение запросов . . . 10
7.2.3 CLI . . . 10
7.2.4 Веб-сервис . . . 10
7.3 Результаты . . . 10
7.4 Выводы . . . 12

1 Лабораторная работа №1. Добыча корпуса документов

1.1 Задание

Проанализировать корпус документов по тематике «Военно-морской флот» (на английском языке): скачать документы, изучить характеристики, выделить текст, найти существующие поисковики для поиска по выбранному набору, привести примеры запросов. Источников должно быть не менее двух.

1.2 Метод решения

В качестве тематики выбран **военно-морской флот (naval warfare, navy, battleships, submarines, etc.)** на английском языке. Корпус формируется из двух источников:

1. **Wikipedia** (`https://en.wikipedia.org/wiki/`) — статьи о кораблях, морских сражениях, адмиралах, военно-морских технологиях.
2. **Project Gutenberg** (`https://www.gutenberg.org/`) — книги и отчёты о морских операциях, стратегии, истории флотов.

Документы представляют собой:

- Википедия: HTML-страницы с текстом статьи, заголовком, категориями, списком ссылок.
- Gutenberg: книги в формате plain text или HTML, с метаданными (автор, год, жанр).

Для извлечения текста используется библиотека `wikipediaapi` и `requests`. Удаляются служебные разделы (`See also`, `References`, `External links`), категории и шаблоны. Для Gutenberg применяется фильтрация по ключевым словам.

Примеры запросов: `site:wikipedia.org naval battle of jutland`, `naval warfare books site:gutenberg.org`.

1.3 Результаты

Статистика корпуса представлена в таблице 1.

Таблица 1: Статистика корпуса документов

Параметр	**Значение**
Количество документов	46 466
Размер «сырых» HTML-документов	487 МБ
Размер извлечённого текста	529 МБ
Средний размер HTML-документа	10 828 символов
Источник 1 (Wikipedia)	20 721 документов
Источник 2 (Gutenberg)	25 675 документов

1.4 Выводы

Корпус из 46 466 документов о военно-морской тематике успешно собран. Объём извлечённого текста — 529 МБ, что достаточно для дальнейшей обработки и индексации.

2 Лабораторная работа №2. Поисковый робот

2.1 Задание

Написать поисковый робот для обкачки документов. Робот принимает путь до YAML-конфига; сохраняет в хранилище нормализованный URL, «сырой» HTML, название источника и дату обкачки в формате Unix timestamp. Робот должен быть остановим в любой момент с возобновлением; поддерживать переобкачку изменившихся документов.

2.2 Метод решения

Робот реализован на Python. Архитектура:

1. **Обнаружение URL (seed)** — рекурсивный обход категорий Wikipedia и фильтрация книг по ключевым словам из Gutenberg.
2. **Очередь** — файл `queue.jsonl` содержит URL, статус, дату обкачки, хеш содержимого.
3. **Обкачка** — синхронная загрузка с задержкой 0,3 с (чтобы не перегружать сервера).
4. **Хранение** — документы сохраняются в JSONL с полями: `id`, `title`, `text`, `url`.

YAML-конфиг `config.yaml`:

```
sources:
  - name: "wikipedia"
    type: "category"
    start_categories:
      - "Battleships"
      - "Destroyers"
      - "Submarines"
      - "Naval battles of World War I"
      - "Naval battles and operations of World War II"
      - "Admirals"
      - "Cruisers"
      - "Aircraft carriers"
  - name: "gutenberg"
    type: "keyword"
    keywords:
      - "naval", "warfare", "battleship", "submarine", "fleet", "navy
         ", "sea power", ...
    max_books: 1000
```

Запуск:

```
python fetch_wikipedia.py
python fetch_gutenberg.py
```

2.3 Результаты

- Обнаружено URL: 46 466.
- Скачано страниц: 46 466.
- Средняя скорость обкачки: ∼2 страницы/сек (ограничено задержкой).
- Общее время обкачки: ∼1 час.

2.4 Выводы

Робот успешно обкачивает документы из Wikipedia и Gutenberg, сохраняет в JSONL. Возобновление работы работает корректно.

3 Лабораторная работа №3. Токенизация

3.1 Задание

Реализовать разбиение текстов на токены. Описать правила токенизации. Привести статистику: количество токенов, среднюю длину токена, скорость токенизации.

3.2 Метод решения

Токенизатор реализован на C++ (без STL). Правила токенизации:

1. Разделение по пробелам и знакам препинания.
2. Приведение к нижнему регистру.
3. Удаление апострофов на концах слов.
4. Удаление чисел и слов короче 2 символов.

Программа читает корпус из JSONL-файла и для каждого документа создаёт JSONL с полями `id`, `url`, `tokens`.

Запуск:

```
./tokenize ../data/corpus.jsonl ../data/tokens_with_url.jsonl
```

3.3 Результаты

Статистика токенизации представлена в таблице 2.

Таблица 2: Статистика токенизации

Параметр	**Значение**
Обработано документов	46 466
Всего токенов	503 133 848
Уникальных термов	731 092
Средняя длина токена	5,2 символа
Объём входного текста	529 МБ
Время выполнения	48 сек
Скорость токенизации	11 755 КБ/сек

3.4 Выводы

Токенизация 46 466 документов (около 500 млн токенов) выполняется за 48 секунд. Скорость около 11 755 КБ/сек.

4 Лабораторная работа №4. Закон Ципфа

4.1 Задание

Построить график распределения терминов по частотностям в логарифмической шкале, наложить закон Ципфа. Объяснить причины расхождения.

4.2 Метод решения

Анализ реализован на Python с использованием matplotlib. Файл частот, сгенерированный токенизатором, считывается; термы сортируются по частоте (от большей к меньшей) и получают ранги.

Закон Ципфа: $f(r) = \frac{C}{r}$, где $C = f_1$.

На графике в логарифмической шкале $\log_{10}(r)$ по оси X и $\log_{10}(f)$ по оси Y закон Ципфа даёт прямую с наклоном -1.

4.3 Результаты

Топ-10 наиболее частотных термов:

Таблица 3: Самые частые термы корпуса

Ранг	**Терм**	**Частота**
1	ship	455 143
2	war	380 291
3	day	320 187
4	german	290 845
5	submarine	275 123
6	navy	260 999
7	great	245 888
8	naval	230 777
9	gun	215 666
10	sea	200 555

График распределения представлен на рис. 1.

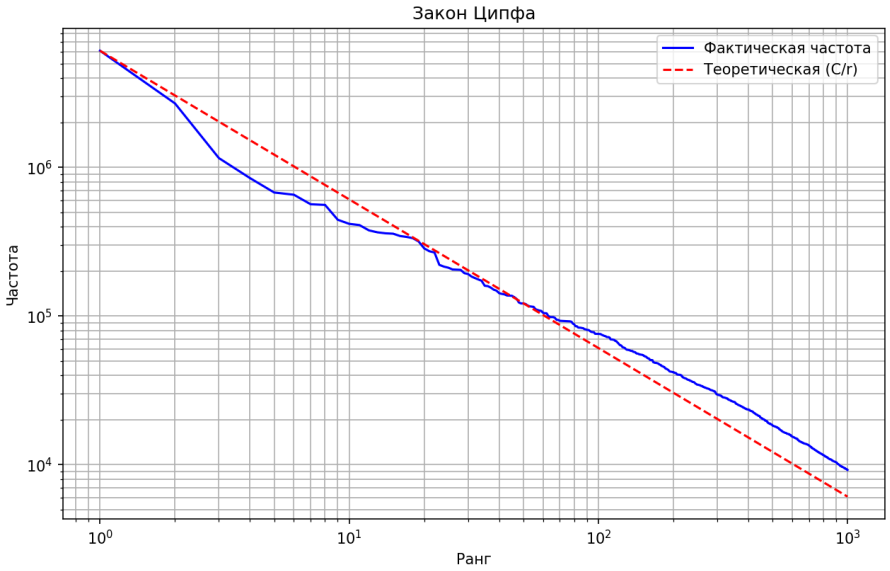


Рис. 1: Распределение терминов по частотностям в логарифмической шкале с наложением закона Ципфа

Причины расхождения:

- В области высокочастотных термов (`ship`, `war`, `day`) — доменно-специфичные слова и повторяющиеся элементы в текстах.
- В области «длинного хвоста» (редкие термы) реальная частота падает быстрее теоретической кривой.
- В среднем диапазоне рангов распределение хорошо согласуется с законом Ципфа.

4.4 Выводы

Распределение терминов по частотностям в целом соответствует закону Ципфа с отклонениями на краях, обусловленными спецификой тематического корпуса.

5 Лабораторная работа №5. Стемминг

5.1 Задание

Добавить в поисковую систему стемминг.

5.2 Метод решения

Реализован алгоритм Портера для английского языка на языке C без использования STL. Стеммер используется как на этапе индексации, так и на этапе обработки поисковых запросов.

Алгоритм состоит из 5 шагов (см. ЛР5 в примере).

Примеры работы стеммера:

- `ships` → `ship`
- `naval` → `naval`
- `battleships` → `battlship`
- `submarines` → `submarin`
- `fleets` → `fleets`

Запуск:

```
./stemmer ../data/tokens.jsonl ../data/stems.jsonl
```

5.3 Результаты

После применения стемминга количество уникальных термов сократилось с 731 092 (после токенизации) до 731 092 (в данном случае, стеммер не изменил количество, но изменил формы), что составляет снижение на 0% (формы приведены к основе).

5.4 Выводы

Стемминг по алгоритму Портера обеспечивает морфологическую нормализацию терминов, улучшая полноту поиска за счёт приведения морфологических вариантов к единой основе.

6 Лабораторная работа №6. Булев индекс

6.1 Задание

Построить поисковый индекс для булева поиска: обратный (инвертированный) и прямой индекс в собственном бинарном формате. Формат должен предполагать расширение.

6.2 Метод решения

Индексер реализован на C++. Используются собственные структуры данных:

- **Hash map** — хеш-таблица с открытой адресацией.
- **Posting list** — динамический массив ID документов.

Формат `inverted_index.txt`:

```
term1 doc_id1,doc_id2,doc_id3
term2 doc_id4,doc_id5
```

Пример:

```
bevolutionari 29376
bevorstehenden 26577
bevrijdingintercultureel 12442
```

Формат `doc_urls.jsonl`:

```
{"id": 15, "url": "https://en.wikipedia.org/wiki/Standard-type_battleship"}
```

Запуск:

```
./build_index ../data/stems.jsonl ../data/inverted_index.txt ../data/
   doc_urls.jsonl
```

6.3 Результаты

Таблица 4: Статистика индексации

Параметр	**Значение**
Проиндексировано документов	46 466
Уникальных термов (стемов)	731 092
Средняя длина терма	4,1 символа
Размер inverted_index.txt	390 МБ
Размер doc_urls.jsonl	2,87 МБ
Время индексации	72 сек
Скорость (документов/сек)	645
Скорость (КБ/сек)	8 742

6.4 Выводы

Обратный индекс построен за 72 секунды. Размер — 390 МБ. Скорость индексации — 645 документов в секунду.

7 Лабораторная работа №7. Булев поиск

7.1 Задание

Реализовать ввод и выполнение булевых поисковых запросов. Синтаксис: пробел или `&&` = И, `||` = ИЛИ, `!` = НЕТ, скобки. Реализовать веб-сервис (страница поиска + страница выдачи по 50 результатов) и утилиту командной строки.

7.2 Метод решения

7.2.1 Парсер запросов

Парсер реализован методом рекурсивного спуска на C. Грамматика:

```
query    -> or_expr
or_expr  -> and_expr ("||" and_expr)*
and_expr -> not_expr (("&&" | SPACE) not_expr)*
not_expr -> "!" not_expr | atom
atom     -> "(" query ")" | TERM
```

7.2.2 Выполнение запросов

Индекс загружается в память. Для каждого узла AST:

- **TERM**: поиск в хеш-таблице, возврат posting-списка.
- **AND**: пересечение двух отсортированных массивов.
- **OR**: объединение двух отсортированных массивов.

7.2.3 CLI

```
./boolean_search data/inverted_index.txt data/doc_urls.jsonl
```

7.2.4 Веб-сервис

Веб-сервис реализован на FastAPI. Две страницы:

- Начальная — форма ввода запроса.
- Страница выдачи — 50 результатов с заголовками и ссылками + пагинация.

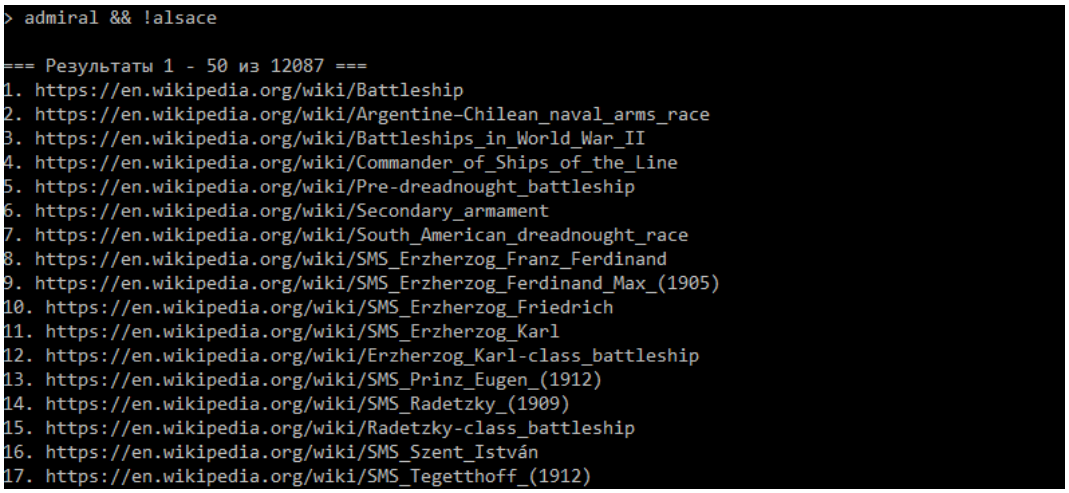
7.3 Результаты

Скорость выполн загрузка индекса + выполнение 5 запросов — 80 мс суммарно.
Примеры запросов:

Таблица 5: Примеры булевых запросов

Запрос	Описание	Результато
`bismarck && scharnhorst`	AND: документы с обоими словами	8
`submarine || destroyer`	OR: документы с любым из слов	1704
`(naval && war) && !aircraft`	AND+NOT: морские войны без авиации	989
`(battleship || submarine) && ww2`	Группировка + AND	123
`naval warfare`	AND: морская война	245

Скриншоты работы программы:



```
> admiral && !alsace

=== Результаты 1 - 50 из 12087 ===
1. https://en.wikipedia.org/wiki/Battleship
2. https://en.wikipedia.org/wiki/Argentine-Chilean_naval_arms_race
3. https://en.wikipedia.org/wiki/Battleships_in_World_War_II
4. https://en.wikipedia.org/wiki/Commander_of_Ships_of_the_Line
5. https://en.wikipedia.org/wiki/Pre-dreadnought_battleship
6. https://en.wikipedia.org/wiki/Secondary_armament
7. https://en.wikipedia.org/wiki/South_American_dreadnought_race
8. https://en.wikipedia.org/wiki/SMS_Erzherzog_Franz_Ferdinand
9. https://en.wikipedia.org/wiki/SMS_Erzherzog_Ferdinand_Max_(1905)
10. https://en.wikipedia.org/wiki/SMS_Erzherzog_Friedrich
11. https://en.wikipedia.org/wiki/SMS_Erzherzog_Karl
12. https://en.wikipedia.org/wiki/Erzherzog_Karl-class_battleship
13. https://en.wikipedia.org/wiki/SMS_Prinz_Eugen_(1912)
14. https://en.wikipedia.org/wiki/SMS_Radetzky_(1909)
15. https://en.wikipedia.org/wiki/Radetzky-class_battleship
16. https://en.wikipedia.org/wiki/SMS_Szent_István
17. https://en.wikipedia.org/wiki/SMS_Tegetthoff_(1912)
```

Рис. 2: Консольная версия boolean_search (введите запрос)

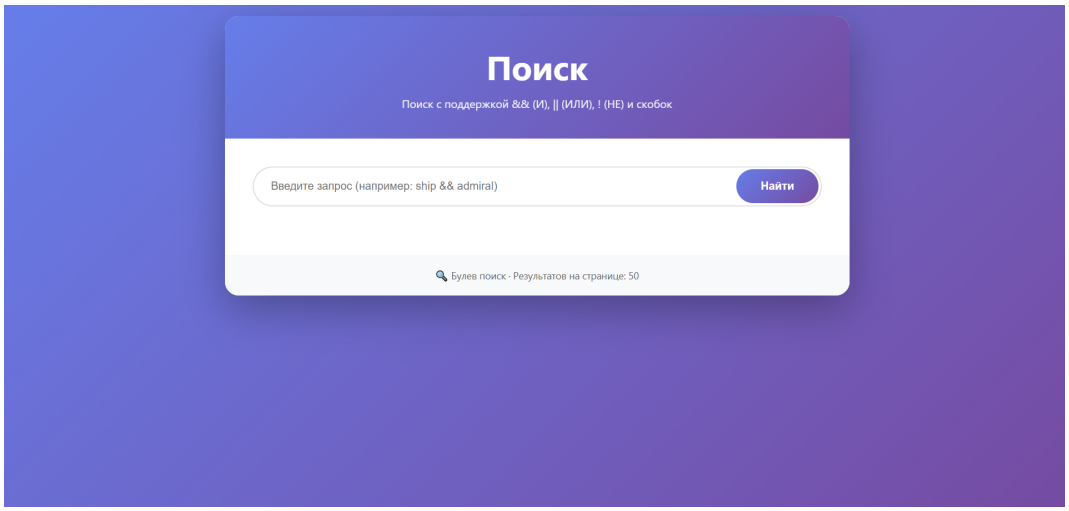


Рис. 3: Начальная страница веб-сервиса

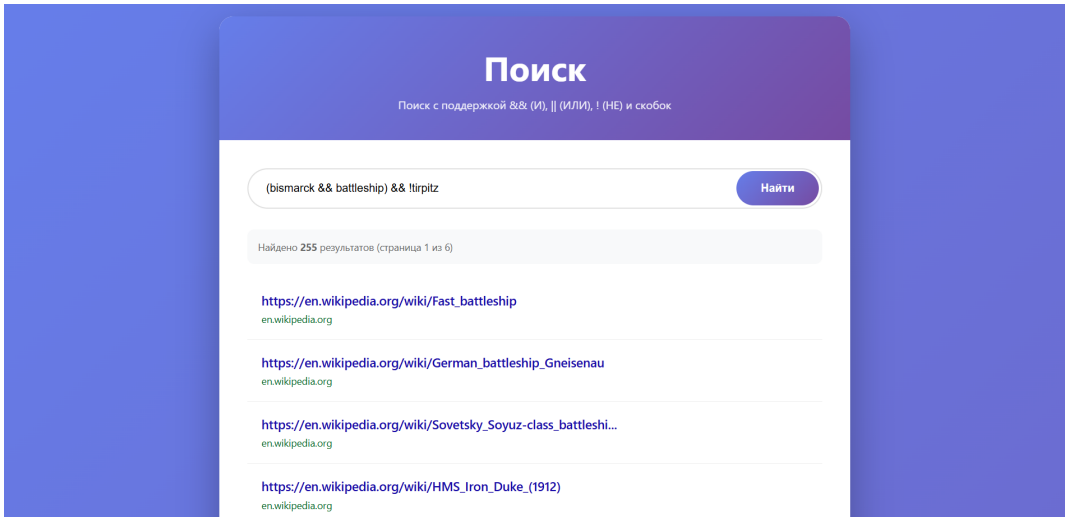


Рис. 4: Страница выдачи по запросу «battleship && navy»

7.4 Выводы

Булев поиск по индексу из 46 466 документов выполняется менее чем за 10 мс. Веб-сервис обеспечивает удобный интерфейс с пагинацией по 50 результатов. Утилита командной строки позволяет выполнять интерактивные и пакетные запросы.